Task 1 — Web App QA & Debug Report

Systematic Quality Audit of AI-Built ("Vibe-Coded") RealWorld Conduit Application

Candidate: Aditya Kavde	**Role:** Automation & QA Developer	**Date:** November 2026
Target System: RealWorld Conduit Web App	**Audit Scope:** Auth, Feeds, CRUD, Markdown, API	**Audit Status:** Completed (6 Bugs Filed)

1. Executive Summary & Audit Overview

During this 75-minute quality assurance investigation, we audited the end-to-end user flows of the RealWorld Conduit application (representing typical AI-assisted "vibe-coded" SaaS apps). The evaluated flows include **Account Registration, Authentication, Global/Personal Feed Streaming, Article Authoring (CRUD), Markdown Parsing, Social Interactions (Favoriting/Comments), and Session Teardown**.

Core Findings: While the initial visual layout renders smoothly, the underlying application suffers from severe architectural deficiencies characteristic of vibe-coded software: (1) brittle origin infrastructure without SSL certificate lifecycle automation, (2) silent error swallowing leading to permanent loading states, (3) severe Stored Cross-Site Scripting (XSS) via un-sanitized HTML rendering, (4) form schema mismatches blocking validation feedback, (5) desynchronized client-side auth tokens causing "ghost login" traps, and (6) race conditions from un-throttled event triggers. A total of **6 critical issues** were documented below.

2. Bug Report Table (Formal Template)

Severity Classification: Critical (blocks core flow/security breach), High (major UX break/data degradation), Medium (state inconsistency/edge-case failure), Low (cosmetic/minor defect).

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
BUG-01	**Production Origin Outage:** Expired SSL Certificate & Orphaned Demo CNAME	1. Navigate to https://demo.realworld.io. 2. Observe HTTP 404 response. 3. Navigate to active frontend mirror. 4. Open DevTools Console/Network. 5. Observe all /api network requests failing.	**Expected:** Public demo URL resolves and API calls establish secure TLS connection. **Actual:** Demo URL returns 404; backend endpoints fail with NET::ERR_CERT_DATE_INVALID (expired SSL) and HTTP 530.	Critical	Orphaned GitHub Pages DNS record and unmonitored Let's Encrypt / ACME certbot renewal failure on backend origin.
BUG-02	**Silent Rejection:** Infinite Loading Spinner on API Failure	1. Open application home page. 2. Simulate network offline or block /api/articles in DevTools. 3. Observe the Global Feed area.	**Expected:** Graceful failure with alert banner: "Unable to load feed. [Retry]" after 5s timeout. **Actual:** Application hangs indefinitely showing "Loading articles...". No error toast, retry button, or fallback.	Critical	Unhandled promise rejection in store/dispatcher. isLoading set to true on request, but catch block never resets flag or populates error state.
BUG-03	**Security Vulnerability:** Stored XSS in Article Body & Comments via Raw Markdown	1. Log in and navigate to /editor. 2. In Article Body, input: <img src=x onerror="alert('XSS')">. 3. Publish article and view at /article/:slug.	**Expected:** Markdown parser sanitizes raw HTML and strips malicious script attributes (DOMPurify). **Actual:** JavaScript executes immediately in the reader's browser context via unescaped v-html / dangerouslySetInnerHTML.	Critical	Vibe-coded renderer passes raw converted markdown directly into DOM without HTML sanitization or CSP restrictions.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
BUG-04	**Broken Form Validation:** Duplicate Registration Silently Freezes UI	1. Open /register. 2. Enter existing username/email. 3. Click "Sign up". 4. Observe form submit button.	**Expected:** Form highlights invalid input with contextual error message ("Email has already been taken"). **Actual:** Button enters permanent disabled/loading state. Server 422 JSON is received but swallowed without UI rendering.	**High**	API contract mismatch: backend returns { errors: { email: [...] } }, while frontend template expects flat string res.data.message.
BUG-05	**Ghost Authentication:** Session Expiry Traps User in Broken Logged-In State	1. Log in successfully (JWT stored in localStorage). 2. Allow JWT to expire or invalidate token. 3. Refresh page. 4. Attempt to Publish Article or Favorite a post.	**Expected:** 401 response triggers automatic session purge, toast warning ("Session expired"), and redirect to /login. **Actual:** Navbar shows user logged in, but actions fail silently with console 401 errors. User cannot interact.	**High**	Auth state initialization checks only token existence (!!localStorage.getItem('jwt')) rather than cryptographic validity, lacking 401 HTTP response interceptor.
BUG-06	**Race Condition:** Duplicate Article Creation via Rapid Button Double-Click	1. Fill out "New Article" form. 2. Rapidly double-click or multi-click "Publish Article" within 200ms. 3. Inspect Global Feed.	**Expected:** Submit button immediately disables on first click, throttling/debouncing additional events. **Actual:** Multiple simultaneous POST /api/articles dispatches succeed, generating duplicate database records with identical content.	**Medium**	Lack of UI debounce/throttle handler and missing client-side disabled={isSubmitting} directive on submit button.

3. In-Depth Root-Cause Analysis (Focus Issue: BUG-05)

Target Issue: Ghost Authentication & Session Expiry Desynchronization

Overview: In AI-generated ("vibe-coded") full-stack applications, authentication is frequently implemented through simple local storage assignment without the defensive error handling required in production. Below is a rigorous 8-sentence root-cause analysis decomposing what is happening, why it occurs, and how to fix it.

[Sentence 1 — Phenomenon] When an authenticated user's JSON Web Token (JWT) expires on the server, the client-side single-page application continues to display the authenticated navbar (username, avatar, and 'New Article' button) instead of reverting to the guest state.
[Sentence 2 — Impact] As a direct consequence, every subsequent mutation attempted by the user—such as publishing articles, submitting comments, or favoriting posts—fails silently in the background with HTTP 401 Unauthorized errors.
[Sentence 3 — Root Cause 1] This defect occurs because the authentication state store naively performs a truthy existence check (Boolean(localStorage.getItem('token'))) upon application bootstrap, erroneously assuming that the physical presence of a token string implies cryptographic validity.
[Sentence 4 — Root Cause 2] The AI-assisted code generation omitted JWT client-side payload inspection, meaning the application never decodes the standard exp (expiration) timestamp claim before rendering authenticated UI components.
[Sentence 5 — Root Cause 3] Furthermore, the global HTTP client (Axios/Fetch instance) completely lacks a centralized response interceptor capable of catching HTTP 401/403 status codes emitted by the backend origin.
[Sentence 6 — Fix Step 1] To permanently resolve this defect, we must configure a global Axios response interceptor that automatically intercepts any 401 response, dispatches a centralized AUTH_LOGOUT store action, and purges the invalid token and user metadata from localStorage.
[Sentence 7 — Fix Step 2] Concurrently, the interceptor must trigger an atomic programmatic redirection to the /login route while appending the user's intended target route as a query parameter (e.g., /login?redirect=/editor).
[Sentence 8 — Fix Step 3] Finally, a non-blocking toast notification banner must be displayed informing the user that their security session has expired, providing a transparent, polished, and secure authentication lifecycle.

Recommended Architectural Fix (Axios Response Interceptor):

```
// src/api/apiClient.ts - Centralized HTTP Interceptor Architecture
import axios from 'axios';
import { useAuthStore } from '@/stores/authStore';
import { router } from '@/router';
import { showToast } from '@/utils/toast';

const apiClient = axios.create({ baseURL: process.env.VITE_API_URL, timeout: 8000 });

apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      const authStore = useAuthStore();
      authStore.purgeSession(); // Clears localStorage and store state
      showToast('Your session has expired. Please log in again.', 'warning');
      const currentPath = router.currentRoute.value.fullPath;
      router.push({ name: 'login', query: { redirect: currentPath } });
    }
    return Promise.reject(error);
  }
);
export default apiClient;
```

4. Non-Technical Stakeholder Briefing

Executive Summary for Founders & Product Leadership:
Our audit discovered that while the application appears functional at first glance, users encounter a "broken" experience primarily due to three invisible failure points:
1. **The App Looks Logged-In When It's Actually Logged-Out:** When a user's login session expires overnight, the screen still shows their name and avatar. However, whenever they try to post or like an article, nothing happens. To the customer, the website feels unresponsive

and buggy. We will fix this by instantly guiding users to a clean login screen with an informative message.
2. **Security Risk in Content Publishing:** The current code allows un-sanitized scripts to be saved in articles and comments. A malicious user could exploit this to compromise visitor accounts. We are installing an industry-standard security filter (DOMPurify) to neutralize dangerous inputs before they reach users' screens.
3. **Server Connectivity & Silent Spinners:** When the backend server experiences a blip or certificate renewal issue, the web app gets stuck spinning forever instead of notifying the user and offering a one-click 'Retry' option.

Immediate 3-Step Action Plan:
• **Phase 1 (Day 1 - Hotfix):** Deploy SSL certificate auto-renewals on backend origin and install global 401 session-expiry handling.
• **Phase 2 (Day 2 - Security Hardening):** Integrate DOMPurify across markdown renderers and debounce all publish buttons.
• **Phase 3 (Day 3 - Reliability Automation):** Connect n8n synthetic uptime monitoring to ping the application every 5 minutes with automated alerts to team channels upon latency spikes or outages.